

CHAPITRE V : ARCHITECTURE ET CHOIX TECHNOLOGIQUES

Les chapitres précédents ont décrit ce que la plateforme doit faire : suivre les demandes de permission, les déclarations de naissance et de décès et les notes de service, chacune avec ses acteurs et son circuit. Ce chapitre explique comment elle est construite. Nous partons du modèle conceptuel pour arriver à la base de données réelle, puis nous justifions le choix du système de gestion de base de données et celui du langage de développement. Nous terminons par l'environnement dans lequel l'application a été réalisée.

I. ÉLABORATION DU MODÈLE PHYSIQUE DES DONNÉES

1. Définition et concepts

Le modèle physique des données, ou MPD, est la dernière étape de la modélisation des données en méthode Merise. Le modèle conceptuel (MCD) décrit les informations et leurs liens sans se soucier de la technique. Le modèle logique (MLD) le traduit en tables. Le modèle physique va plus loin : il décrit comment ces tables sont réellement créées dans le système de gestion de base de données choisi. On y trouve le nom exact de chaque table et de chaque colonne, le type de chaque donnée, les valeurs obligatoires ou facultatives, les clés, les index et les contraintes qui protègent la cohérence des données.

Quelques notions reviennent tout au long de ce chapitre :

- **La table** regroupe les lignes qui décrivent un même type d'objet. Elle correspond à une entité du MCD, par exemple agents ou demandes_permission.
- **La colonne** est une information de la table, avec son type : nombre entier, texte, date ou booléen. Ainsi nombre_jours est un entier et date_debut une date.
- **La clé primaire** identifie chaque ligne de façon unique. Elle vient de l'identifiant du MCD, et toutes nos tables ont une colonne id.
- **La clé étrangère** référence la clé primaire d'une autre table. Elle traduit une association du MCD : agent_id, dans demandes_permission, désigne l'agent qui a déposé la demande.
- **La contrainte d'unicité** interdit deux lignes avec la même valeur, comme deux agents de même matricule.
- **L'index** accélère les recherches sur une colonne. Le statut d'une demande en a un, car il sert à chaque affichage de liste.
- **L'intégrité référentielle** empêche de référencer une ligne qui n'existe pas et fixe ce qui se passe à la suppression.

Pour passer du MCD au MPD, nous avons appliqué les règles habituelles de Merise, résumées à la figure 1. Chaque entité devient une table, et son identifiant devient la clé primaire. Une

association dont les cardinalités sont 0,n d'un côté et 1,1 de l'autre disparaît en donnant une clé étrangère dans la table du côté 1,1. L'association DEMANDER entre l'agent et la demande de permission produit ainsi la colonne agent_id dans demandes_permission. Quand la cardinalité est 0,1, la clé étrangère est facultative : c'est le cas de gestionnaire_id, qui reste vide tant que le gestionnaire RH n'a pas traité la demande.

Une association qui possède ses propres attributs les confie à la table qui porte la clé. Les attributs avis_gestionnaire et date_verif_rh de l'association VÉRIFIER se retrouvent donc dans demandes_permission. Une association de type plusieurs à plusieurs devient une table de liaison : role_user relie les comptes à leurs rôles, et note_structure relie une note de service aux structures destinataires. Enfin, une association réflexive, comme le lien de dépendance entre structures, donne une clé étrangère vers la même table : structures.parent_id.

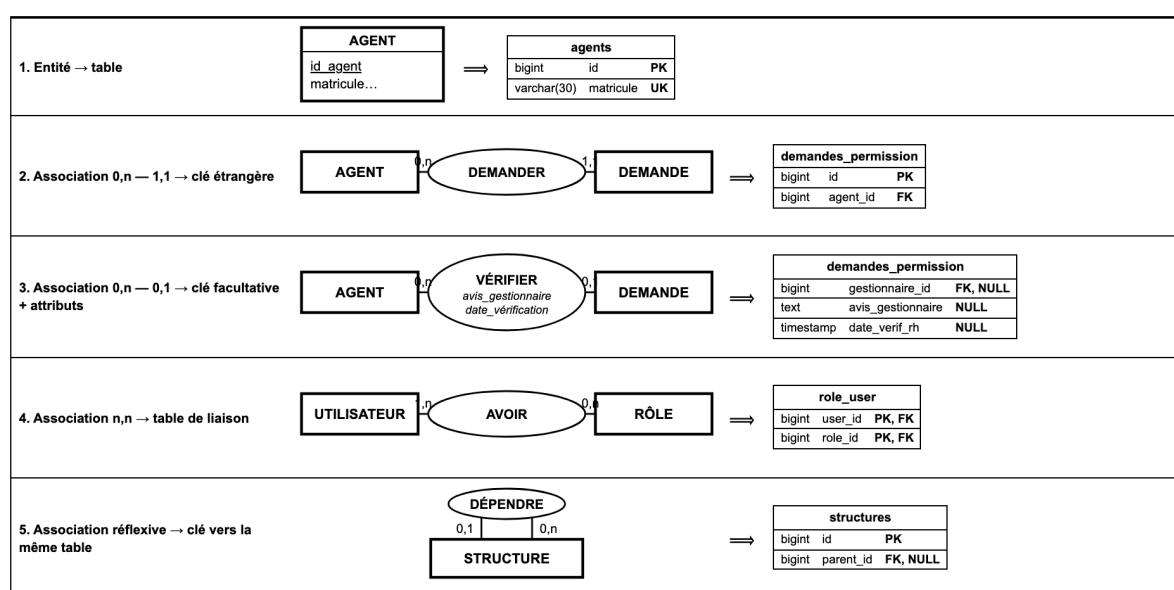


Figure 1 : Règles de passage du modèle conceptuel au modèle physique

2. Le modèle physique de la plateforme

La base de données s'appelle gfp_laravel. Elle contient 30 tables : 21 tables métier, qui portent les données de l'application, et 9 tables techniques créées par le cadre de développement pour ses propres besoins (suivi des migrations, sessions, cache, files de tâches, jetons d'accès, réinitialisation des mots de passe). Les tables métier se répartissent en six domaines.

Domaine	Tables
Organisation, comptes et droits	agents, structures, fonctions, users, roles, privileges, role_user, role_privilege
Permissions	types_permission, demandes_permission, demande_historique
État civil	declarations_naissance, declarations_decès, declaration_historique
Notes de service	notes_service, note_structure, note_historique
Pièces et alertes	pieces_jointes, notifications

La figure 2 donne la vue d'ensemble du modèle : les 21 tables métier avec leurs clés primaires, leurs clés étrangères et leurs colonnes à valeur unique, ainsi que les 35 liens qui les relient. Le trait part de la clé primaire de la table parent et arrive, avec une patte d'oie, sur la clé étrangère de la table enfant. Un cercle sur le trait indique que la clé étrangère est facultative. Le détail de toutes les colonnes est donné domaine par domaine en annexe.

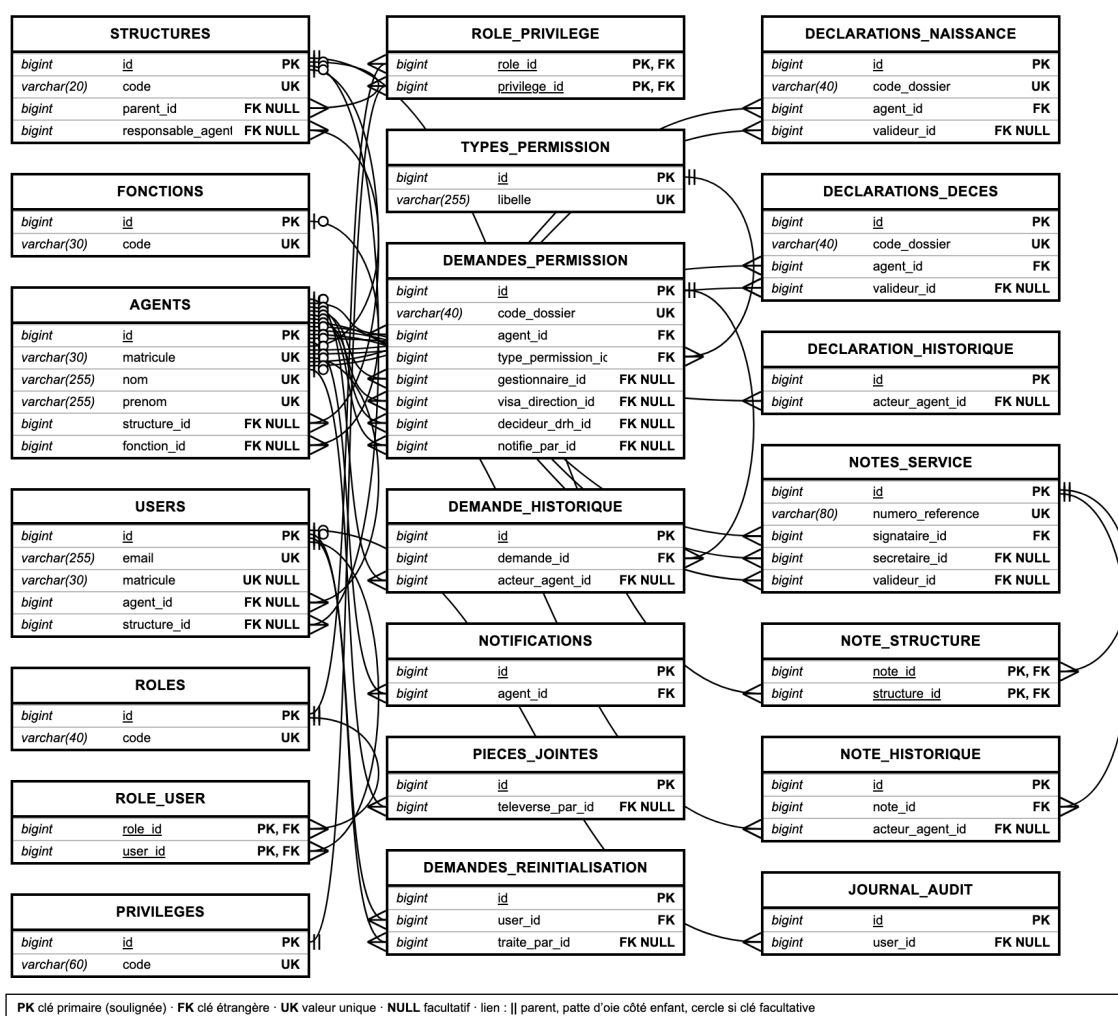


Figure 2 : Modèle physique des données, vue d'ensemble (clés et liens)

Prenons la table demandes_permission, représentée en détail à la figure 3. Elle porte tout le circuit d'une permission. Elle contient les informations saisies par l'agent : le type de permission, les dates de début et de fin, le nombre_jours (de 1 à 30), le motif et le chemin du justificatif. Elle contient aussi ce que produisent les trois acteurs qui interviennent : l'avis du gestionnaire RH, le visa du Sous-Directeur ou du Directeur, et la décision du DRH, chacun avec sa date. Le statut indique l'étape actuelle : en attente du gestionnaire RH, du visa ou du DRH, retour pour correction, validée ou rejetée. Le code_dossier, unique, donne la référence lisible du dossier, par exemple PERM-2026-XT8HOZ.

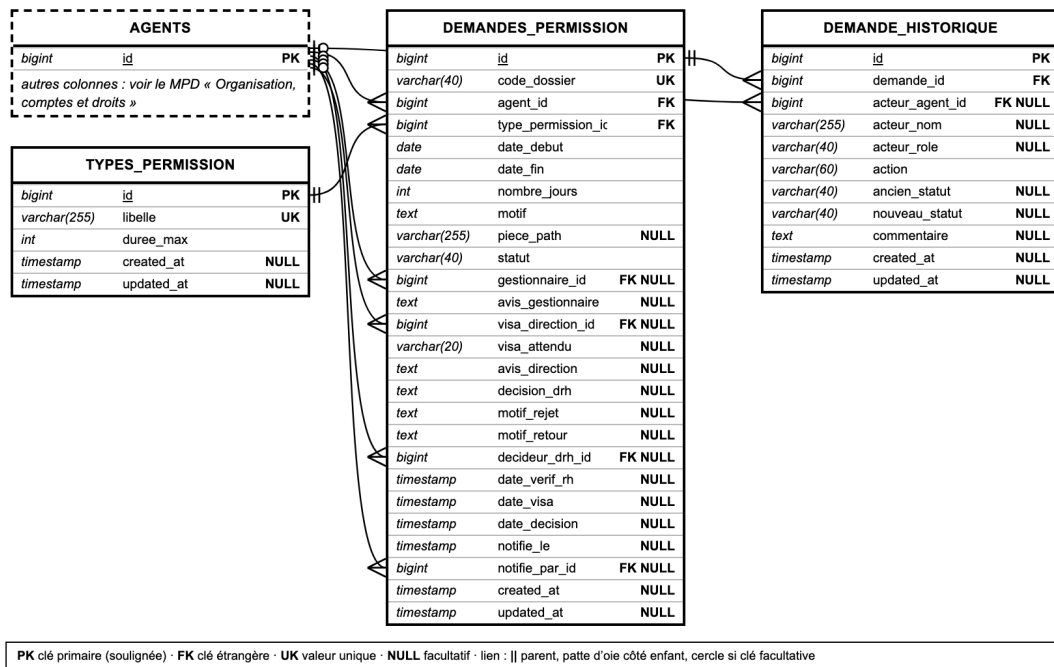


Figure 3 : Modèle physique des demandes de permission

Plusieurs règles sont confiées directement à la base, pour que la cohérence ne dépende pas seulement du code de l'application. Les colonnes matricule, code_dossier et numero_reference sont uniques. Un agent est aussi unique par le couple nom et prénom, ce qui empêche de créer deux fois la même personne. Les clés étrangères suivent trois comportements. Pour les 20 clés facultatives, la suppression de la ligne référencée met simplement la clé à vide : supprimer un agent ne fait pas disparaître les dossiers qu'il a traités. Pour les 13 clés des tables de liaison et des historiques, la suppression entraîne celle des lignes dépendantes. Deux clés seulement interdisent la suppression tant qu'elles sont utilisées.

Certaines données ne sont jamais stockées en clair. Les mots de passe, les jetons d'accès et les codes de réinitialisation sont enregistrés sous forme hachée. Les justificatifs ne sont pas dans la base : elle ne garde que leur chemin, et les fichiers restent dans un stockage privé que seul le serveur peut lire, après vérification des droits de la personne qui les demande.

Le modèle est créé par 17 fichiers de migration, exécutés dans l'ordre par une seule commande. Ils permettent de reconstruire la base à l'identique sur n'importe quelle machine. Un jeu de données de départ charge les structures du ministère, les rôles et des comptes de démonstration, et un export SQL complet est conservé dans le dépôt du projet.

II. CHOIX DE SYSTÈME DE GESTION DE BASE DE DONNÉES

Nous avons comparé les systèmes de gestion de base de données selon les besoins réels de la plateforme. Elle sera utilisée en même temps par des agents de plusieurs directions. Elle manipule des dossiers officiels dont l'historique doit rester fiable, et elle doit pouvoir

s'installer sans budget de licence. Nous avons donc retenu sept critères : le modèle relationnel, les accès simultanés, les transactions, le coût, la compatibilité avec Laravel, la présence d'un outil d'administration simple, et la disponibilité chez les hébergeurs.

Critère	SQLite	MySQL / MariaDB	PostgreSQL	SQL Server / Oracle
Fonctionnement	simple fichier	serveur	serveur	serveur
Accès simultanés	limités en écriture	prévus pour de nombreux utilisateurs	prévus pour de nombreux utilisateurs	prévus pour de nombreux utilisateurs
Transactions	oui	oui (moteur InnoDB)	oui	oui
Licence	libre, gratuit	libre, gratuit	libre, gratuit	payante
Outil d'administration	peu répandu	phpMyAdmin, très répandu	pgAdmin	outils propriétaires
Hébergement	petits usages	proposé par presque tous les hébergeurs	moins répandu	coûteux
Adéquation au projet	bien pour les essais, insuffisant en production	adaptée	adaptée, plus lourde à administrer ici	disproportionnée

Tableau 1 : Comparaison des systèmes de gestion de base de données

Nous avons retenu **MariaDB**, un système de gestion de base de données relationnel libre, issu de MySQL et compatible avec lui. Il répond à tous les critères : il gère sans difficulté plusieurs utilisateurs à la fois, son moteur InnoDB garantit les transactions et l'intégrité référentielle décrite plus haut, il est gratuit et il fonctionne avec Laravel par le pilote MySQL. Il s'administre avec **phpMyAdmin**, une interface web qui permet de consulter les tables, de lancer des requêtes et d'exporter la base. Nous utilisons MariaDB 13.0.2 et phpMyAdmin 5.2.3.

Le projet a d'abord été développé avec SQLite, dont la base tient dans un simple fichier et ne demande aucune installation, ce qui a permis d'avancer vite au début. Nous sommes ensuite passés à MariaDB pour travailler dans des conditions proches d'une vraie mise en service, avec un serveur et plusieurs connexions en parallèle. SQLite reste utilisé pour les tests automatiques : chaque test s'exécute sur une base en mémoire, créée puis détruite, ce qui les rend rapides et indépendants les uns des autres.

La base utilise le moteur InnoDB et le jeu de caractères utf8mb4, qui permet d'enregistrer sans problème les accents et les apostrophes. L'application s'y connecte avec un compte dédié, distinct du compte administrateur, dont les droits sont limités à cette base.

III. CHOIX DE LANGAGE DE DÉVELOPPEMENT

1. Côté serveur : PHP et Laravel

La partie serveur est écrite en **PHP**, avec le cadre de développement **Laravel**. PHP est un langage conçu pour le web : très répandu, gratuit, pris en charge par tous les hébergeurs et bien documenté. Nous utilisons la version 8.5, qui permet de typer les paramètres et les retours des fonctions. Cela évite des erreurs dans des règles aussi sensibles que celles des circuits de validation.

Laravel, dans sa version 13, organise le code selon le modèle MVC (modèle, vue, contrôleur) et apporte de nombreux outils que nous aurions dû écrire nous-mêmes autrement :

- **les routes** définissent toutes les adresses de l'API : dépôt d'une demande, avis du gestionnaire RH, décision du DRH, diffusion d'une note ;
- **les intergiciels** vérifient la connexion, contrôlent le rôle exigé pour chaque route et limitent les tentatives de connexion à dix par minute ;
- **Eloquent**, le mappage objet-relationnel, lit et écrit les tables sous forme d'objets (Agent, DemandePermission, NoteService) et écrit les requêtes SQL à notre place ;
- **les migrations** décrivent la base de façon versionnée ;
- **la validation** contrôle les données reçues : durée de 1 à 30 jours, motif obligatoire, justificatif au bon format ;
- **Sanctum** gère l'authentification par jeton d'accès personnel, révoqué à la déconnexion ;
- **PHPUnit** exécute les tests automatiques.

Nous avons rangé les règles des circuits dans des services distincts : un pour les permissions, un pour l'état civil et un pour les notes de service. Les contrôleurs se contentent de recevoir la demande, de valider les données et de répondre. Cette séparation permet de tester chaque règle isolément et de la modifier sans toucher au reste. Les droits reposent sur un contrôle d'accès par rôle : chaque route déclare les rôles autorisés, et les règles plus fines sont vérifiées dans les services. Par exemple, seul le responsable de la structure d'un agent peut donner le visa sur sa demande.

2. Côté navigateur : HTML, CSS et JavaScript

L'interface est écrite en **HTML5**, avec **Tailwind CSS** pour la mise en forme et du **JavaScript** natif pour le comportement des pages. Nous n'avons pas utilisé de cadre JavaScript comme React ou Vue. Les écrans de la plateforme sont des listes, des formulaires et des tableaux, pour lesquels un cadre lourd n'apporterait rien. Cette simplicité rend l'application plus légère à charger, sans étape de compilation, et plus facile à maintenir. Chaque acteur dispose d'une page adaptée à son rôle, avec un en-tête commun qui affiche son identité, sa structure et ses notifications. Les échanges avec le serveur se font par des requêtes au format JSON. Les

graphiques du tableau de bord statistique sont dessinés avec la bibliothèque **Chart.js**, dans sa version 4.4.4.

3. Architecture générale et sécurité

La plateforme suit une architecture en trois couches, représentée à la figure 4. Le navigateur envoie des requêtes au serveur. Le serveur applique les intergiciels, valide les données, exécute les règles métier, puis lit ou écrit dans la base par l'intermédiaire des modèles. Il s'appuie aussi sur un stockage privé pour les justificatifs et sur une messagerie pour l'envoi des notes de service.

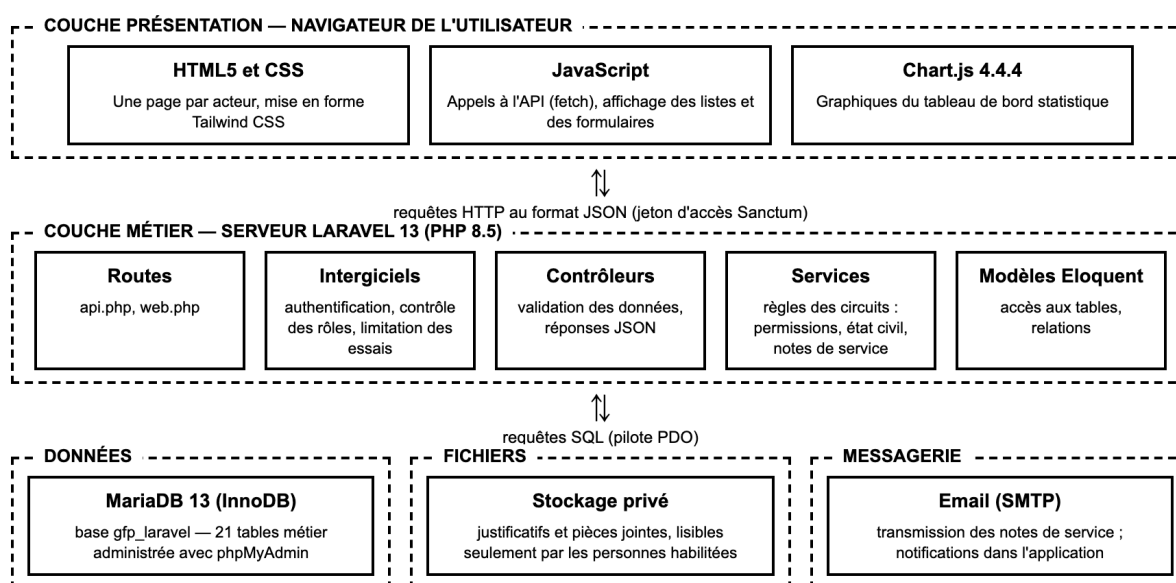


Figure 4 : Architecture technique de la plateforme

La sécurité a été prise en compte à chaque niveau. Les mots de passe sont hachés et ne sont jamais visibles, y compris pour l'administrateur. Toute donnée affichée est échappée pour éviter l'injection de code dans les pages. Chaque action est contrôlée côté serveur selon le rôle de l'utilisateur, sans se fier à ce que l'interface affiche. Les justificatifs ne sont accessibles qu'aux personnes habilitées. Un journal d'audit conserve les connexions, les déconnexions, les tentatives refusées et les actions sensibles, avec l'adresse et l'heure. Enfin, l'administrateur peut suspendre un compte : la personne est alors déconnectée et ne peut plus se connecter.

Couche	Technologie	Version
Base de données	MariaDB (moteur InnoDB)	13.0.2
Administration de la base	phpMyAdmin	5.2.3
Langage serveur	PHP	8.5
Cadre de développement	Laravel	13
Authentification	Laravel Sanctum	4.3
Interface	HTML5, Tailwind CSS, JavaScript	—

Graphiques	Chart.js	4.4.4
Tests	PHPUnit	12.5

Tableau 2 : Technologies utilisées

IV. ENVIRONNEMENT DE DÉVELOPPEMENT

La plateforme a été développée sur un ordinateur portable Apple fonctionnant sous macOS. Tous les outils ont été installés localement avec le gestionnaire de paquets Homebrew, ce qui donne un environnement facile à reproduire sur une autre machine. Le code a été écrit avec l'éditeur Visual Studio Code. Pendant le développement, l'application s'exécute sur le serveur intégré de Laravel, à l'adresse locale sur le port 8000, avec PHP 8.5 et Composer pour installer les bibliothèques PHP. La base de données tourne sur MariaDB 13.0.2, consultée avec phpMyAdmin sur le port 8081. Node.js et npm servent aux outils de mise en forme du code, et Laravel Pint veille au respect d'un style commun. La figure 5 résume cet environnement.

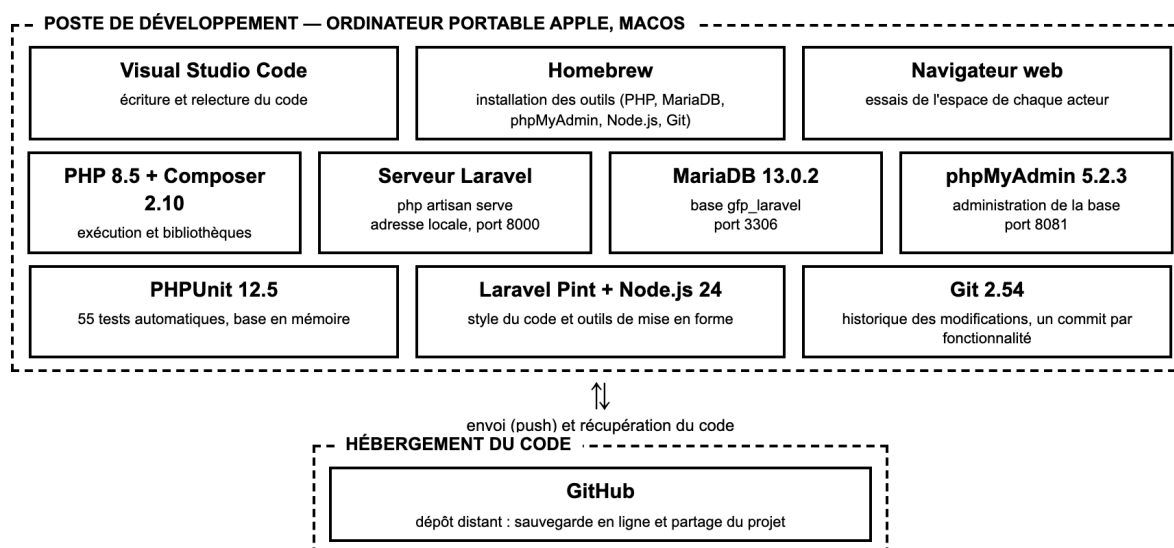


Figure 5 : Environnement de développement

Le code est versionné avec **Git** et sauvegardé en ligne sur **GitHub**. Chaque fonctionnalité a fait l'objet d'un commit, ce qui conserve tout l'historique des modifications et permet de revenir en arrière en cas de problème.

Le code est organisé de façon classique pour un projet Laravel :

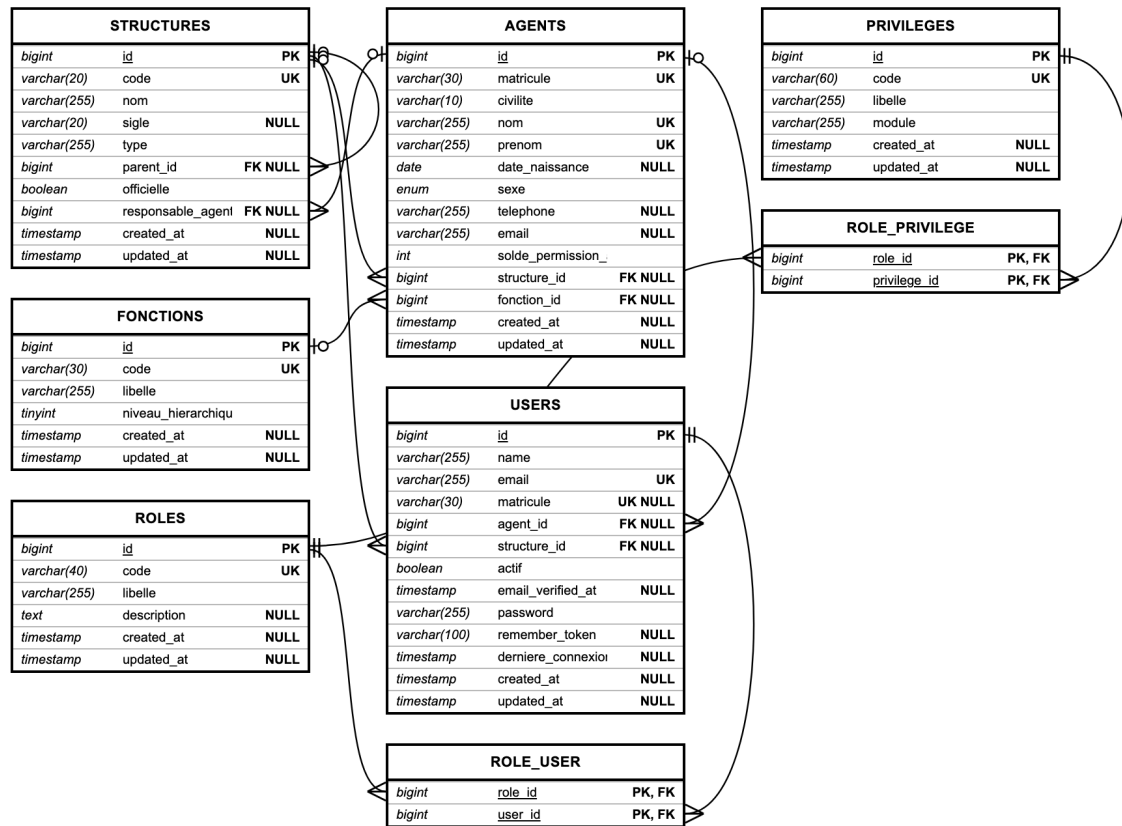
- le dossier app contient les contrôleurs, les modèles, les services qui portent les règles des circuits et les notifications ;
- le dossier database regroupe les migrations, les jeux de données de départ et l'export SQL de la base ;
- le dossier routes contient les adresses de l'API ;

- le dossier public contient l'interface des différents espaces ;
- le dossier tests contient les tests automatiques ;
- le dossier docs conserve la documentation de conception, avec les modèles Merise.

Les tests automatiques vérifient les circuits complets et les droits de chaque rôle. Ils sont au nombre de 55 et contrôlent 279 points. Par exemple, ils vérifient qu'une demande de plus de deux jours passe directement du gestionnaire RH au DRH, qu'un Directeur ne peut pas viser la demande d'un agent d'une autre structure, ou qu'un compte suspendu ne peut plus se connecter. Ils tournent en quelques secondes et sont relancés après chaque modification importante.

Pour installer la plateforme sur une autre machine, il suffit de récupérer le dépôt, d'installer les dépendances avec Composer, de renseigner dans le fichier de configuration l'accès à la base de données, puis de lancer la création des tables et le chargement des données de départ. Le serveur de développement démarre ensuite avec une seule commande, et l'application est accessible depuis le navigateur.

Les figures suivantes détaillent toutes les colonnes de chaque table, domaine par domaine. Le domaine des permissions est présenté à la figure 3. Les tables représentées en pointillés ne montrent que leur clé primaire : elles sont décrites dans le premier domaine.



PK clé primaire (soulignée) · FK clé étrangère · UK valeur unique · NULL facultatif · lien : || parent, patte d'oie côté enfant, cercle si clé facultative

Figure A1 : Organisation, comptes et droits

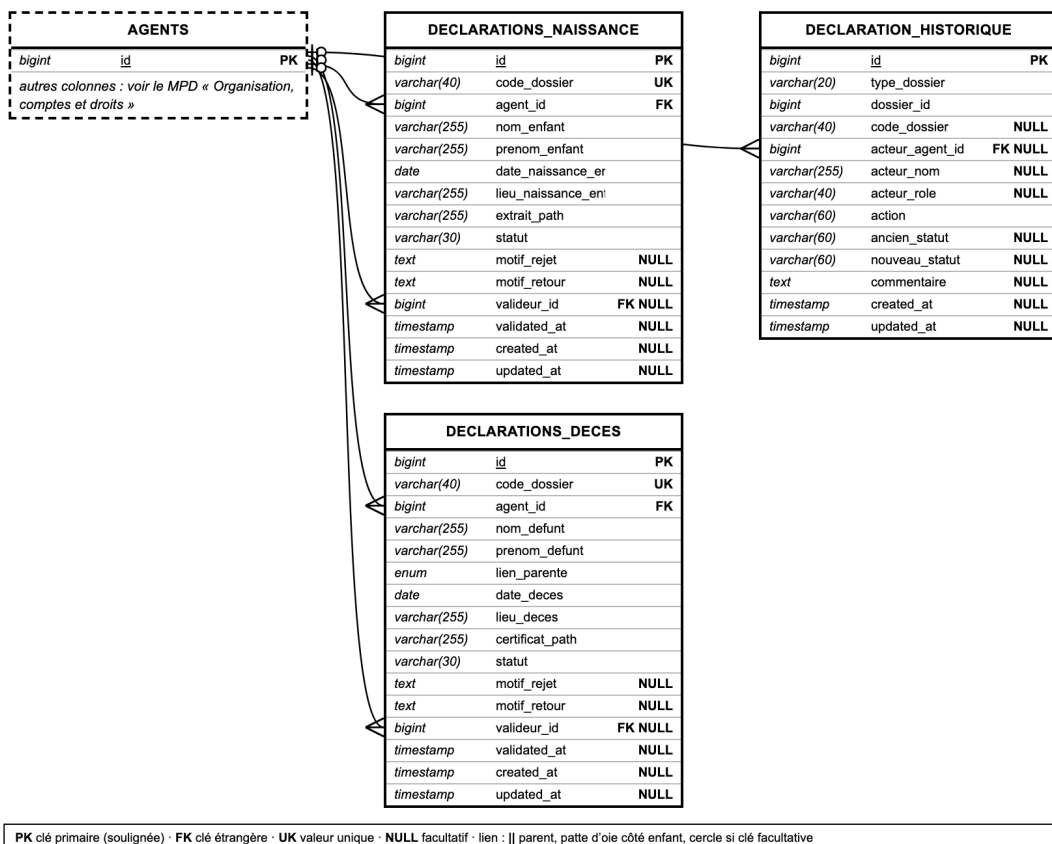


Figure A2 : Déclarations de naissance et de décès

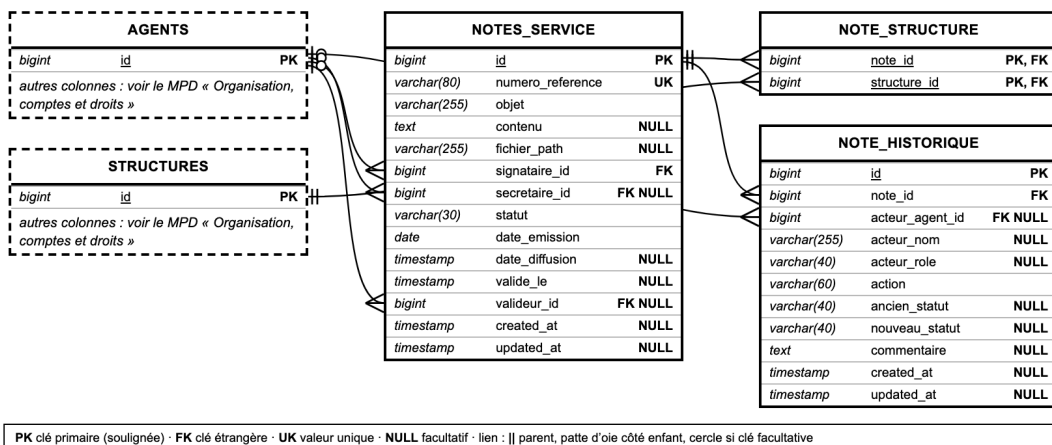
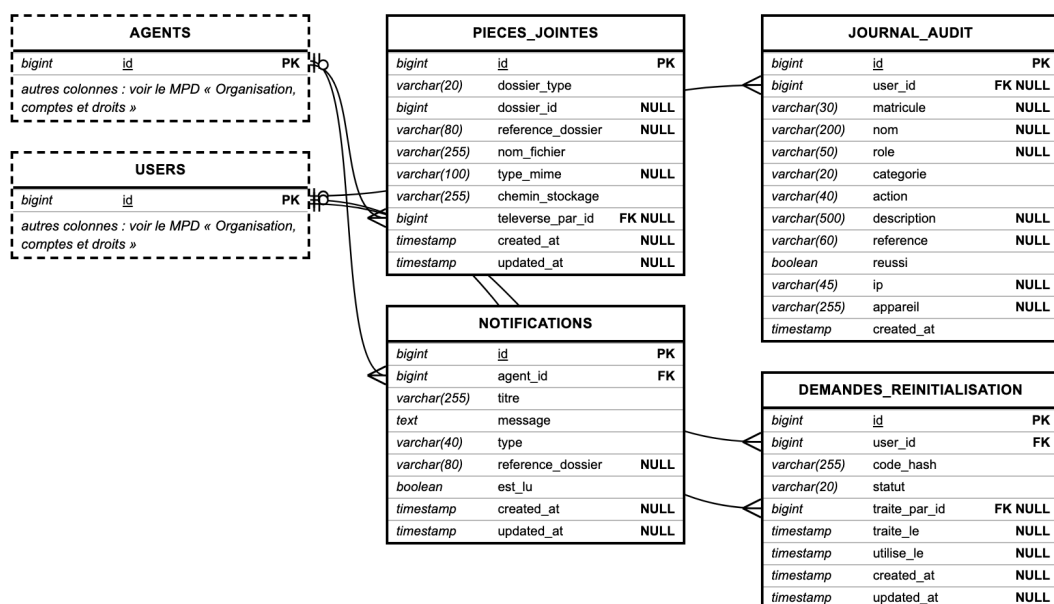


Figure A3 : Notes de service



PK clé primaire (soulignée) · FK clé étrangère · UK valeur unique · NULL facultatif · lien : || parent, patte d'oie côté enfant, cercle si clé facultative

Figure A4 : Pièces jointes, notifications et sécurité